基于 Spark Structured Streaming 的实时热门统计系统

BITMZ043 云计算与大数据分析 课程实践报告

方向 C：基于 Spark 的实时数据分析

姓名：赵英杰 **学号**：D23090600756

姓名：韦多展 **学号**：D23090600792

2026 年 4 月 5 日

摘要

本项目基于 Apache Spark Structured Streaming 框架，结合 Apache Kafka 消息队列，构建了一个面向电商场景的实时热门统计系统。系统实现了实时热门商品 Top-N 排行和实时热搜词统计两大核心功能，采用滑动窗口聚合与 Watermark 机制处理乱序和迟到数据。同时，本项目实现了基于 Spark DataFrame API 的批处理分析，从延迟、吞吐量等维度对流处理与批处理进行了定量对比。实验结果表明，流处理可在秒级延迟内输出实时统计结果，而批处理在吞吐量方面更具优势，两者适用于不同的业务场景。本报告详细阐述了系统的架构设计、关键技术原理（DAG 执行机制、内存计算、惰性求值）、数据处理流程、性能优化手段及容错机制，并通过 Streamlit 可视化仪表盘展示实时分析结果。

关键词：Spark Structured Streaming；Kafka；实时计算；滑动窗口；流批对比

项目代码：https://github.com/Ludan-daye/realtime-hot-stats

目录

1 项目背景 **4**

1.1 课题背景 4

1.2 项目目标 4

1.3 技术栈概述 5

2 系统架构设计 **5**

2.1 整体架构 5

2.2 Kafka 消息设计 6

2.3 Docker 容器化部署 7

3 关键技术原理 **8**
3.1 DAG 执行机制 8
3.2 内存计算（Memory-based Computing） 9
3.3 惰性求值（Lazy Evaluation） 9

4 数据处理流程 **10**
4.1 数据生成与发送 10
4.1.1 加权随机模型 10
4.1.2 迟到数据模拟 11
4.2 Spark Streaming 处理流程 11
4.2.1 Watermark 机制 12
4.2.2 滑动窗口 12
4.2.3 foreachBatch 输出模式 12
4.3 批处理流程 13

5 实验结果与分析 **13**
5.1 实验环境 13
5.2 流处理实验结果 13
5.2.1 实时热门商品 Top-10 13
5.2.2 实时热搜词 Top-10 14
5.3 批处理实验结果 14
5.3.1 热门商品 Top-10（全量统计） 15
5.3.2 商品分类浏览量 15
5.4 流处理 vs 批处理性能对比 16
5.5 批处理各阶段耗时 17

6 性能优化手段 **17**
6.1 Spark 层面优化 17
6.2 Kafka 层面优化 18
6.3 数据层面优化 18

7 容错机制 **18**
7.1 Checkpoint 恢复 18
7.2 容错演示流程 18

8 可视化展示 **19**

9 总结与展望 **21**
9.1 项目总结 21
9.2 技术反思 22
9.3 未来展望 22

A 团队分工 **23**

B 项目代码结构 **23**

1 项目背景

1.1 课题背景

随着互联网和移动互联网的快速发展，电商平台每天产生海量的用户行为数据。据统计，大型电商平台在促销期间每秒可产生数十万条用户浏览、搜索和购买事件。如何从这些持续产生的数据流中实时提取有价值的信息——如当前最热门的商品、用户最关注的搜索词——已成为电商运营和决策的关键需求。实时热门统计在电商运营中具有多方面的应用价值：运营团队可以据此实时调整首页推荐位和广告投放策略；供应链系统可以基于热门趋势提前备货，减少缺货风险；营销团队可以在热点出现的第一时间策划促销活动，最大化流量转化率。

传统的批处理方式（如 Hadoop MapReduce）需要等待数据收集完毕后才能开始处理，通常以小时甚至天为周期运行，存在较高的延迟，无法满足实时分析的需求。MapReduce 的计算模型要求每次 Map 和 Reduce 之间的中间数据都写入 HDFS 磁盘，在迭代计算和多阶段流水线场景下性能损耗尤为严重。而流式计算框架（如 Apache Spark Structured Streaming）通过微批次（Micro-Batch）或连续处理（Continuous Processing）模式，能够在数据到达后秒级输出分析结果，为实时监控和即时决策提供了技术支撑。

随着”摩尔定律”的逐渐失效，单机计算能力的增长放缓，分布式并行编程已成为处理海量数据的必然选择 [4]。Apache Spark 作为新一代大数据计算引擎，凭借其内存计算、统一的编程模型和丰富的生态系统，已成为学术界和工业界广泛采用的分布式计算框架 [1]。Spark Structured Streaming 在 Spark SQL 引擎之上提供了声明式的流处理 API，使得开发者可以像编写批处理查询一样编写流处理逻辑，大幅降低了流式应用的开发门槛 [2]。

1.2 项目目标

本项目选择方向 C（基于 Spark 的实时数据分析），目标是构建一个完整的实时热门统计系统，具体包括：

1. **实时热门商品统计**：基于滑动窗口实时计算浏览量 Top-10 商品。
2. **实时热搜词统计**：实时统计用户搜索频次最高的关键词。
3. **流批对比分析**：对同一数据集分别进行流处理和批处理，从延迟、吞吐量等维度进行定量对比。
4. **可视化展示**：通过 Streamlit 仪表盘实时展示分析结果。
5. **容错机制**：基于 Checkpoint 实现流处理的故障恢复。

1.3 技术栈概述

本项目涉及的分布式技术栈如表 1 所示，覆盖了分布式存储、分布式计算、资源调度和数据接口四个方面。本项目的完整源代码托管于 GitHub 仓库：https://github.com/Ludan-daye/realtime-hot-stats，采用 Git 进行版本控制，便于团队协作和代码审查。

表 1: 项目技术栈

技术领域	组件	版本	作用
消息队列	Apache Kafka	3.9.0	分布式数据接口，解耦生产者与消费者
流处理引擎	Spark Structured Streaming	3.5.5	分布式计算，微批次流处理
编程语言	Python (PySpark)	3.13	应用开发
容器编排	Docker Compose	—	资源调度，环境一键部署
可视化	Streamlit	1.45	实时仪表盘展示

2 系统架构设计

2.1 整体架构

系统采用经典的 Lambda 架构思想，同时支持实时流处理路径（Speed Layer）和批处理路径（Batch Layer），如图 1 所示。Lambda 架构由 Nathan Marz 提出，其核心理念是将数据处理分为三层：批处理层负责对全量历史数据进行精确计算，速度层负责对最新增量数据进行低延迟近似计算，服务层负责合并两者的结果对外提供查询。本系统在此基础上进行了简化，保留了速度层（Spark Streaming）和批处理层（Spark Batch）的双路径设计，并通过 Streamlit 仪表盘作为统一的服务层展示两路结果。

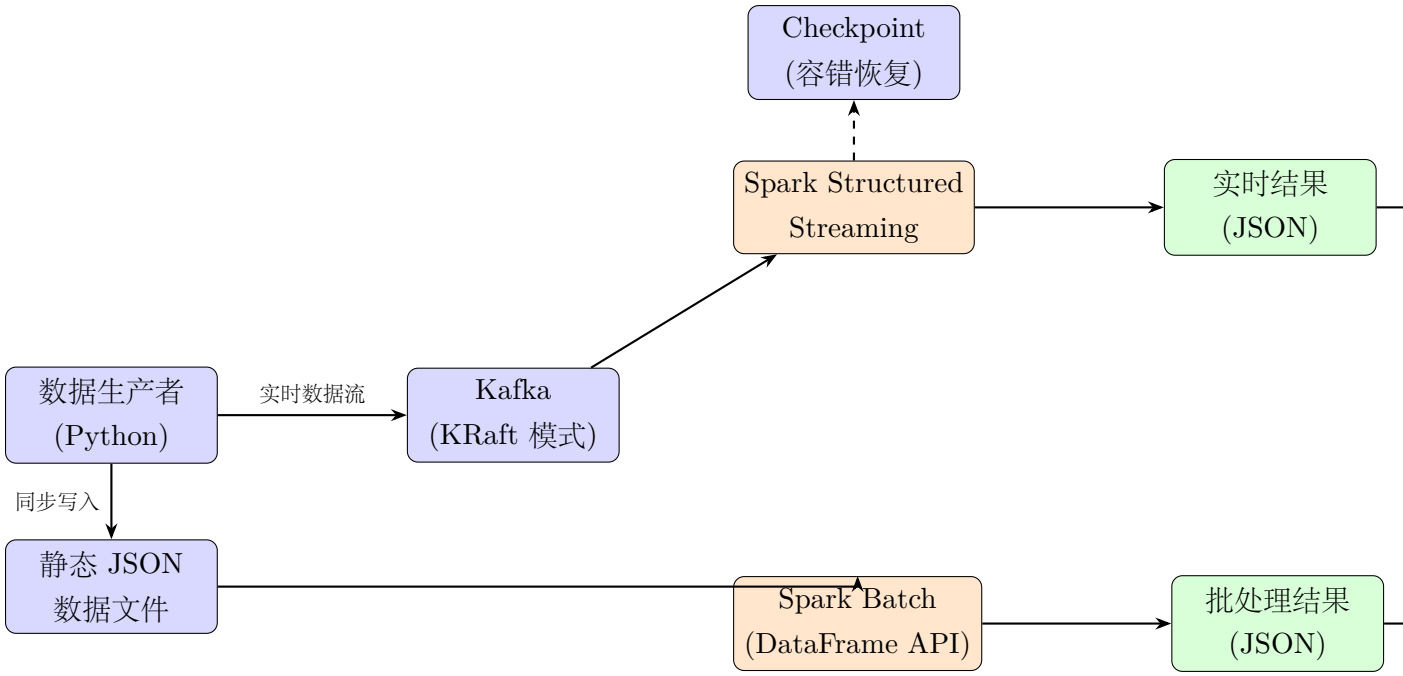

图 1: 系统整体架构图

架构核心特点：

- **数据源层**：Python 数据生产者使用加权随机（Zipf 分布）模拟用户浏览和搜索行为，通过 Kafka Producer API 将事件写入 Kafka Topic，同时将数据写入本地 JSON 文件供批处理使用。
- **消息中间件层**：Kafka 采用 KRaft 模式（无 Zookeeper），作为流数据的缓冲和分发中心，解耦数据生产与消费。
- **计算层**：Spark Structured Streaming 从 Kafka 订阅实时数据流进行滑动窗口聚合；Spark Batch 读取静态 JSON 文件进行全量分析。
- **输出与展示层**：计算结果以 JSON 格式写入文件系统，Streamlit 仪表盘定时读取并可视化展示。

2.2 Kafka 消息设计

系统定义了两个 Kafka Topic，分别承载不同类型的用户行为事件：

表 2: Kafka Topic 设计

Topic 名称	分区数	消息 Schema
`user_browse`	3	`user_id`, `product_id`, `product_name`, `category`, `event_type`, `timestamp`, `producer_timestamp`
`user_search`	3	`user_id`, `keyword`, `platform`, `timestamp`, `producer_timestamp`

其中 `timestamp` 为事件时间（Event Time），用于窗口聚合；`producer_timestamp` 为消息发送时间，用于计算端到端延迟。两个字段的分离使得系统能够模拟迟到数据以验证 Watermark 机制。

消息设计遵循了以下原则：（1）**事件溯源**：每条消息包含完整的事件上下文（用户、商品/关键词、时间、平台等），无需回查外部系统即可完成聚合计算；（2）**Schema 显式定义**：在 Spark 端使用 `StructType` 显式定义 Schema，而非使用 `inferSchema`，避免 Schema 推断带来的性能开销和类型不确定性；（3）**分区策略**：每个 Topic 设置 3 个分区，使 Spark 可以分配多个 Task 并行消费，提升读取吞吐量。在生产环境中，分区数应根据消费者数量和数据量进一步调优。

`user_browse` 中的 `event_type` 字段区分了四种用户行为：`view`（浏览）、`click`（点击）、`add_cart`（加购）和 `purchase`（购买），其生成比例为 50:30:15:5，符合电商漏斗模型中从浏览到购买逐级递减的特征。`user_search` 中的 `platform` 字段记录了用户的搜索设备类型（mobile/pc/tablet），可用于后续的多维分析。

2.3 Docker 容器化部署

系统使用 Docker Compose 编排基础设施，实现一键部署：

```
docker compose up -d     # 启动 Kafka (KRaft 模式)
make producer            # 启动数据生产者
make submit-hot-products  # 提交流处理任务
```

Listing 1: 环境启动命令

Kafka 采用 Apache 官方镜像 `apache/kafka:3.9.0`，以 KRaft（Kafka Raft）模式运行。KRaft 是 Kafka 3.0 引入的新型元数据管理协议，使用 Raft 共识算法替代了传统的 ZooKeeper 外部依赖 [3]。在本项目的单节点场景下，KRaft 模式将 Broker 和 Controller 合并在同一进程中运行，显著简化了部署拓扑：无需额外启动 ZooKeeper 容器，减少了资源占用和运维复杂度。配置中通过 `CLUSTER_ID` 环境变量指定集群标识，`KAFKA_PROCESS_ROLES=broker,controller` 使单节点同时承担数据服务和元数据管理职责。

Spark 在本地以 `local[2]` 模式运行，分配 2 个线程模拟 2 个 Worker 核心，利用本机多核并行处理。Spark 连接 Kafka 所需的 `spark-sql-kafka-0-10` 连接器通过 `--packages` 参数在运行时自动下载，无需手动管理 JAR 依赖。整个环境的启动和管理通过 Makefile 封装为一组语义化命令，降低了使用门槛。

3 关键技术原理

本节深入分析 Spark 的三大核心技术原理，这些原理是理解流处理系统性能优势的理论基础。

3.1 DAG 执行机制

Spark 将用户编写的数据处理逻辑编译为一个有向无环图（Directed Acyclic Graph, DAG）。DAG 中的每个节点代表一个 RDD（弹性分布式数据集）或 DataFrame 上的操作，边代表数据依赖关系。

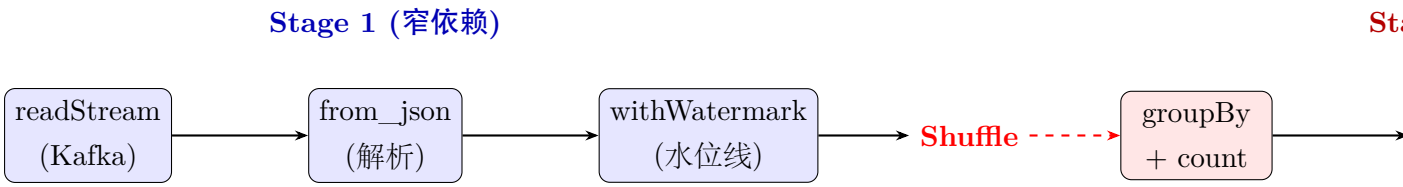

图 2: 热门商品统计的 DAG 执行图

在本项目中，热门商品统计的 DAG 包含两个 Stage：

- **Stage 1**（窄依赖）：从 Kafka 读取数据 → JSON 解析 → 设置 Watermark。这些操作不涉及数据重分布（Shuffle），可以在同一分区内流水线执行。
- **Stage 2**（宽依赖）：`groupBy` 聚合操作触发 Shuffle，将相同 `product_id` 的数据汇聚到同一分区进行计数，然后排序取 Top-10。

Spark 的 DAG 调度器（DAGScheduler）会将 DAG 在 Shuffle 边界处切分为多个 Stage，每个 Stage 内的任务可以并行执行，从而最大化计算并行度。

DAG 执行机制的优势体现在多个层面。首先，**流水线优化**（Pipeline Optimization）：Stage 内部的多个窄依赖操作会被合并为一条流水线，数据在单个 Task 中连续经过所有操作，无需中间物化（Materialization），减少了内存分配和数据序列化的开销。其次，**Shuffle 优化**：Spark 在 Shuffle 阶段使用基于排序的 Shuffle Manager（Sort Shuffle Manager），将同一分区的数据合并写入单个文件，避免了早期 Hash Shuffle 产生大量小文件的问题。在本项目中，由于将 `spark.sql.shuffle.partitions` 设为 4（而非默认的 200），每次 Shuffle 仅产生 4 个分区的数据文件，显著减少了文件系统操作和网络传输量。最后，**Catalyst 优化器**：Spark SQL 的 Catalyst 优化器在生成物理执行计划之前，会对逻辑计划进行多轮优化，包括谓词下推（Predicate Pushdown）、列裁剪

（Column Pruning）、常量折叠（Constant Folding）等。在本项目中，Catalyst 会自动将 Kafka 消息中未使用的字段（如 `key`、`partition`、`offset`）在读取阶段即行裁剪，减少不必要的数据传输。

3.2 内存计算（Memory-based Computing）

Spark 相较于 Hadoop MapReduce 的核心优势在于内存计算。图 3 对比了两种计算模型的数据流：

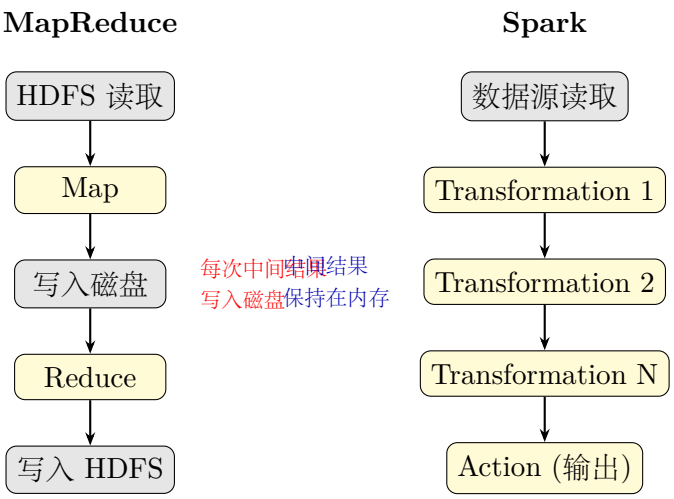

图 3: MapReduce vs Spark 数据流对比

MapReduce 在每个 Map 和 Reduce 阶段之间需要将中间结果写入磁盘（HDFS），而 Spark 将中间结果保持在内存中，仅在必要时（如 Shuffle 或 Checkpoint）才写入磁盘。在本项目的流处理场景中，这一特性体现为：

- 每个微批次的 JSON 解析、Watermark 过滤、窗口聚合均在内存中流水线完成。
- State Store（状态存储）使用 `HDFSBackedStateStoreProvider`，将窗口聚合的中间状态缓存在内存中，仅定期写入 Checkpoint。
- 相比 MapReduce 的”读-算-写-读-算-写”模式，Spark 的”读-算-算-算-写”模式显著减少了磁盘 I/O。

3.3 惰性求值（Lazy Evaluation）

Spark 中的操作分为两类：

1. **Transformation**（转换操作）：如 `select`、`filter`、`groupBy`、`withWatermark` 等。这些操作不会立即执行，而是构建计算图（DAG）。
2. **Action**（行动操作）：如 `start()`（流处理）、`collect()`、`show()`、`write()` 等。只有 Action 触发时，Spark 才会执行整个 DAG。

在本项目代码中，惰性求值的体现如下：

```
# 以下操作均为 Transformation，不会立即执行
raw_df = read_kafka_stream(spark, BROWSE_TOPIC)          # Lazy
parsed_df = raw_df.select(from_json(...))                 # Lazy
windowed = parsed_df.withWatermark(...).groupBy(...).agg(...)  # Lazy

# 只有 start() 是 Action，触发整个 DAG 的执行
query = windowed.writeStream.foreachBatch(...).start()    # 触发执行！
```

Listing 2: 惰性求值示例（hot_products.py）

惰性求值的优势在于：Spark 可以在执行前对整个 DAG 进行全局优化（如谓词下推、列裁剪、Stage 合并），避免不必要的中间计算。

值得注意的是，惰性求值在流处理场景中具有特殊意义。在传统的即时求值（Eager Evaluation）模式下，每一步操作都会立即执行并产生中间结果，这对于流处理来说意味着大量的重复计算。而 Spark 的惰性求值允许引擎将整个流处理管道作为一个完整的逻辑计划来分析和优化，只有当微批次触发（每 10 秒）时才将优化后的物理计划应用于当前批次的数据。这种”先规划后执行”的策略，使得 Spark 能够在保持代码简洁性的同时实现高效的执行性能。

此外，惰性求值还为用户提供了**容错的语义保证**：由于 Transformation 仅描述计算逻辑而不执行计算，Spark 可以在 Task 失败时，根据 DAG 的血缘关系（Lineage）重新计算丢失的分区数据，而无需从头重新执行整个流水线。这一机制是 Spark 实现 Exactly-Once 语义的基础之一。

4 数据处理流程

4.1 数据生成与发送

数据生产者模拟电商平台的用户行为数据，其核心设计包括：

4.1.1 加权随机模型

为模拟真实场景中的”热门效应”（少数商品和关键词占据大量流量），数据生成器采用加权随机抽样。每个商品和关键词预设了一个权重值（1–10），权重越高被选中的概率越大。

例如”无线蓝牙耳机”权重为 10，”电饭煲”权重为 3，前者被用户浏览的概率约为后者的 $10/3 \approx 3.3$ 倍。这种分布近似于 Zipf 分布，符合现实世界中用户行为的长尾特征。

具体而言，数据生成器维护了两个字典文件：`products.json`（50 个商品，涵盖数码、运动、家电、美妆等 15 个分类）和 `search_keywords.json`（40 个搜索关键词）。Python 的 `random.choices` 函数根据预设权重进行加权随机抽样，确保高权重商品和关键词具有更高的出现频率。数据生产者还支持通过命令行参数 `--rate` 动态调节发送速率（默认 100 条/秒），这为后续的性能测试提供了灵活的负载控制能力。浏览事件和搜索事件的发送比例固定为 7:3，反映了电商场景中浏览行为远多于搜索行为的真实特点。

4.1.2 迟到数据模拟

通过 `--late-ratio` 参数控制迟到数据比例（默认 10%）。迟到数据的事件时间（`timestamp`）比实际发送时间早 10–60 秒，用于验证 Watermark 机制的有效性。

```python
if random.random() < late_ratio:
    delay = random.randint(10, 60)
    event_time = now - timedelta(seconds=delay)  # 事件时间回退
else:
    event_time = now
```

Listing 3: 迟到数据模拟

4.2 Spark Streaming 处理流程

流处理的核心逻辑如图 4 所示：

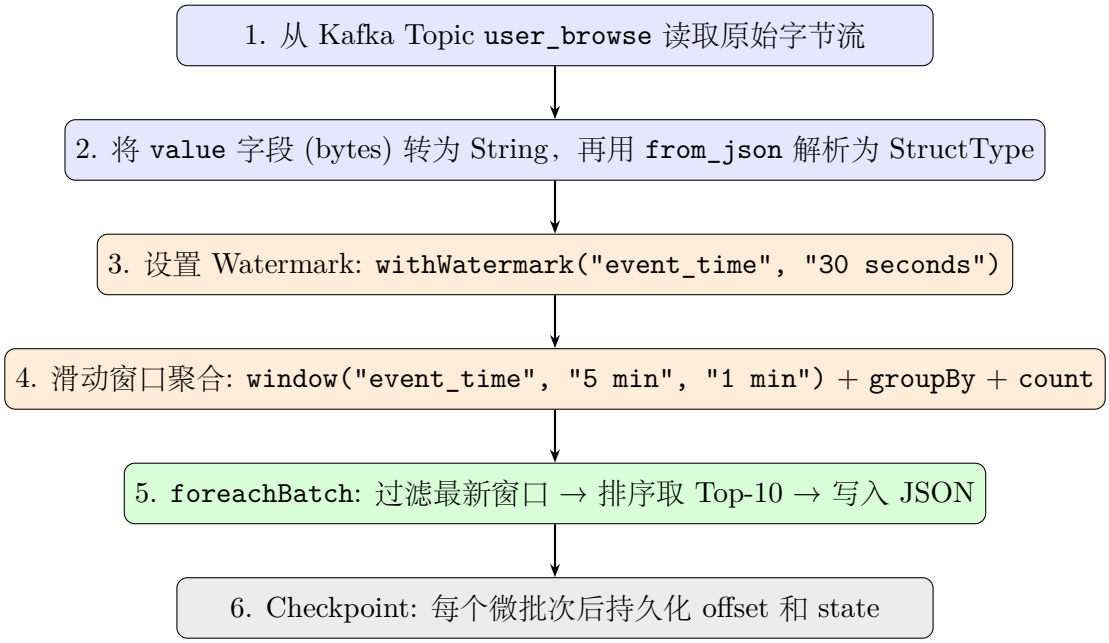

图 4: Spark Streaming 数据处理流程

4.2.1 Watermark 机制

Watermark（水位线）用于处理乱序到达的事件。本项目设置 Watermark 为 30 秒，含义是：系统允许事件时间比当前最大事件时间晚 30 秒以内的数据参与窗口聚合。超过 30 秒的迟到数据将被丢弃，以避免无限期维护旧窗口的状态。

$$\text{Watermark} = \max(\text{event_time}) - \text{threshold} \tag{1}$$

当某个窗口的结束时间 $<$ Watermark 时，该窗口的状态会被清除，结果会被最终输出。

Watermark 的选取需要在**数据完整性**和**状态管理开销**之间权衡。较大的 Watermark 阈值能够容纳更多迟到数据，提升结果的准确性，但代价是需要更长时间维护窗口状态，增加内存消耗和结果输出延迟；较小的阈值可以更快地释放旧窗口状态，降低资源占用，但可能丢弃合理的迟到数据。本项目选择 30 秒作为 Watermark 阈值，这是基于以下考量：数据生成器中迟到数据的最大延迟为 60 秒，10% 的迟到数据比例中，延迟在 10–30 秒之间的数据约占 33%，因此 30 秒的阈值能够覆盖大部分迟到数据，同时将状态维护的时间窗口控制在可接受的范围内。

4.2.2 滑动窗口

本项目采用 5 分钟的窗口大小和 1 分钟的滑动步长，意味着：

- 每条事件会被分配到 5 个重叠的窗口中（因为 $5 \div 1 = 5$）。
- 每分钟产生一个新的统计窗口，保证结果的时效性。
- 窗口聚合通过 Spark 的 State Store 在微批次之间维护中间状态。

4.2.3 foreachBatch 输出模式

由于 Structured Streaming 的 `update` 输出模式不支持直接排序（`orderBy`），本项目使用 `foreachBatch` Sink：每个微批次将聚合结果作为静态 DataFrame 处理，在其上执行排序和 Top-N 操作。

```python
def process_batch(batch_df, batch_id):
    if batch_df.isEmpty():
        return
    latest_window = batch_df.select("window") \
        .orderBy(desc("window")).first()
    top10 = batch_df \
        .filter(col("window") == latest_window["window"]) \
        .orderBy(desc("view_count")) \
        .limit(10)
    top10.coalesce(1).write.mode("overwrite").json(OUTPUT_PATH)
```

Listing 4: foreachBatch 中取 Top-10

4.3 批处理流程

批处理读取 Producer 同步生成的静态 JSON 文件，执行与流处理相同的统计逻辑：

1. 加载全量数据（50,000 条）
2. 分离浏览事件和搜索事件
3. 全量 Top-10 商品统计 + 窗口 Top-10 统计
4. 热搜词 Top-10 统计
5. 商品分类浏览量统计
6. 输出性能报告（各阶段耗时、总吞吐量）

5 实验结果与分析

5.1 实验环境

表 3: 实验环境配置

配置项	规格
操作系统	macOS Darwin 25.2.0 (Apple Silicon)
CPU	Apple M 系列芯片
内存	16 GB
Java	OpenJDK Temurin 17.0.17
Python	3.13.5 (Anaconda)
PySpark	3.5.5
Kafka	3.9.0 (KRaft 模式, Docker)
Spark 运行模式	local[2]

5.2 流处理实验结果

5.2.1 实时热门商品 Top-10

在 Producer 以 50 条/秒的速率发送约 50 秒（共 2,068 条浏览事件）后，Spark Streaming 输出了 5 个微批次的结果。最终批次的 Top-10 热门商品如表 4 所示：

表 4: 流处理热门商品 Top-10（最终批次）

排名	商品名称	分类	浏览量
1	手机壳	配件	75
2	运动鞋	运动	74
3	机械键盘	数码	64

结果与商品字典中的权重分布一致：手机壳（权重 9）、运动鞋（权重 9）、机械键盘（权重 8）位列前三，验证了加权随机数据生成器的正确性。

从实验过程可以观察到，流处理结果随时间动态变化：在第 1 个批次（约 10 秒）时，运动鞋以 15 次浏览量排名第一；到第 3 个批次时，运动鞋增长至 46 次，手机壳紧随其后以 44 次排名第二；到第 5 个（最终）批次时，手机壳以 75 次反超运动鞋（74 次）夺得第一。这种排名的实时波动体现了流处理系统”数据驱动、持续更新”的核心特征，与批处理的”一次性全量计算”形成鲜明对比。值得注意的是，由于滑动窗口的重叠效应，同一商品在不同窗口中的计数值可能存在差异，这正是滑动窗口”多角度观察同一时间段数据”的设计意图。

5.2.2 实时热搜词 Top-10

热搜词统计的结果如表 5 所示：

表 5: 热搜词 Top-10

排名	关键词	搜索次数
1	蓝牙耳机	531
2	运动鞋	491
3	防晒霜	446
4	T 恤男	422
5	智能手表	413
6	手机壳	406
7	保温杯	400
8	机械键盘	390
9	连衣裙夏季	389
10	空气炸锅	378

5.3 批处理实验结果

对 50,000 条数据进行批处理分析，结果如下：

5.3.1 热门商品 Top-10（全量统计）

表 6: 批处理热门商品 Top-10

排名	商品名称	商品 ID	分类	浏览量
1	无线蓝牙耳机	p_001	数码	1,242
2	手机壳	p_005	配件	1,125
3	运动鞋	p_011	运动	1,116
4	空气炸锅	p_015	家电	1,040
5	机械键盘	p_002	数码	1,033
6	T 恤	p_031	服装	1,029
7	降噪耳机	p_008	数码	989
8	防晒霜	p_018	美妆	946
9	充电线	p_045	配件	912
10	保温杯	p_014	生活	910

5.3.2 商品分类浏览量

表 7: 商品分类浏览量分布

分类	浏览量	占比
数码	10,747	30.7%
家电	4,294	12.3%
配件	2,693	7.7%
服装	2,637	7.5%
美妆	2,481	7.1%
运动	2,156	6.2%

数码类商品以 30.7% 的占比遥遥领先，这是因为数码类别下有较多高权重商品（无线蓝牙耳机 10、机械键盘 8、降噪耳机 8 等）。

5.4 流处理 vs 批处理性能对比

表 8: 流处理与批处理性能对比

指标	流处理	批处理
首条结果延迟	~10 秒（首个微批次触发后）	8.25 秒（全量数据处理后）
持续输出间隔	10 秒（微批次触发间隔）	N/A（一次性输出）
处理总耗时	持续运行	8.25 秒
吞吐量	受 Producer 速率限制	6,063 条/秒
数据新鲜度	秒级	分钟/小时级
适用场景	实时监控、告警、仪表盘	离线报表、历史分析
资源占用	持续占用	按需启动

分析:
从实验数据可以得出以下几点关键结论:

- **延迟特性差异显著**: 流处理的首条结果延迟仅为 ~10 秒（取决于微批次触发间隔），且后续结果每 10 秒持续更新。批处理则需要等待 8.25 秒处理完全部 50,000 条数据后才能输出第一条结果。当数据量增长到百万级甚至亿级时，批处理的延迟将线性增长，而流处理的延迟保持稳定在微批次间隔附近，这一差异将更为显著。
- **吞吐量各有侧重**: 批处理在本次实验中达到 6,063 条/秒的吞吐量，因为它可以一次性读取全量数据，充分利用数据局部性和批量 I/O 优化。流处理的吞吐量受限于 Producer 发送速率和微批次间隔设置，但其设计目标并非最大化吞吐量，而是在保证低延迟的前提下提供" 足够好" 的吞吐能力。
- **资源利用模式不同**: 流处理需要持续占用计算资源（CPU、内存），即使在没有新数据到来时也会周期性地检查 Kafka 偏移量。批处理则按需启动，任务完成后释放资源。在资源有限的环境中，可以根据业务对实时性的要求选择合适的处理模式。
- **实际生产中的互补关系**: 在实际生产中，两者通常结合使用（Lambda 架构或 Kappa 架构）。流处理提供实时视图供运营监控和即时决策；批处理定期（如每天凌晨）对全量数据进行重新计算，修正流处理可能因迟到数据或状态管理导致的累积误差，生成精确的离线报表。

5.5 批处理各阶段耗时

表 9: 批处理各阶段耗时明细

阶段	耗时 (秒)	占比
数据加载	3.35	40.6%
热门商品统计	1.12	13.6%
窗口商品统计	2.04	24.7%
热搜词统计	0.48	5.8%
分类统计	0.50	6.1%
总计	**8.25**	**100%**

数据加载阶段占总耗时的 40.6%，是最大的瓶颈。这是因为 JSON 是一种文本格式，解析时需要逐字符扫描、处理转义字符、推断数据类型，计算开销较大。此外，JSON 文件不支持列式读取——即使查询只涉及 `product_id` 和 `product_name` 两个字段，也必须将整行数据完整读入并解析。在生产环境中，可改用 Apache Parquet 等列式存储格式，其优势包括：（1）列式压缩（Column-level Compression）可将存储体积减少 60%–80%；（2）列裁剪（Column Pruning）允许仅读取查询所需的列，减少磁盘 I/O；（3）二进制编码无需字符解析，CPU 开销显著降低。根据业界经验，从 JSON 切换到 Parquet 后，数据加载速度通常可提升 5–10 倍。

窗口商品统计（2.04 秒）耗时高于全量商品统计（1.12 秒），这是因为滑动窗口聚合需要为每条记录分配到 5 个重叠窗口中（窗口大小 5 分钟 / 滑动步长 1 分钟 = 5 个窗口），相当于将数据量膨胀了 5 倍。热搜词统计（0.48 秒）和分类统计（0.50 秒）耗时较短，因为搜索事件数量仅为浏览事件的 3/7，且聚合维度更简单（无窗口操作）。

6 性能优化手段

6.1 Spark 层面优化

1. **Shuffle 分区数调优**：将 `spark.sql.shuffle.partitions` 从默认的 200 调整为 4，适配单机环境的核心数，避免过多小任务的调度开销。

2. **Trigger 间隔设置**：设置 `trigger(processingTime="10 seconds")`，每 10 秒触发一次微批次处理，平衡延迟与吞吐量。

3. **coalesce(1) 输出优化**：在 `foreachBatch` 中使用 `coalesce(1)` 将结果合并为单个文件输出，避免产生大量小文件。

4. **Checkpoint 机制**：为每个流处理任务配置独立的 Checkpoint 目录，确保 Kafka offset 和窗口状态的持久化，支持故障恢复。

6.2 Kafka 层面优化

1. **分区策略**：设置 3 个分区（`KAFKA_NUM_PARTITIONS=3`），使 Spark 能够并行读取，提高消费吞吐量。
2. **KRaft 模式**：采用 Kafka 3.9.0 的 KRaft 模式，去除 Zookeeper 依赖，减少单节点部署的资源开销和运维复杂度。
3. **日志保留策略**：设置 `KAFKA_LOG_RETENTION_HOURS=1`，及时清理过期数据，控制磁盘使用。

6.3 数据层面优化

1. **Producer 批量 flush**：每 100 条消息执行一次 `flush()`，减少网络调用次数，提升发送效率。
2. **双时间戳设计**：事件中同时携带 `timestamp`（事件时间）和 `producer_timestamp`（发送时间），前者用于窗口聚合，后者用于延迟测量。
3. **Watermark 阈值选择**：30 秒的 Watermark 阈值在数据完整性（允许迟到数据）和状态维护开销（及时释放旧窗口）之间取得平衡。

7 容错机制

7.1 Checkpoint 恢复

在分布式系统中，节点故障是常态而非异常。一个健壮的流处理系统必须能够在故障发生后自动恢复，且不丢失数据、不重复处理。Spark Structured Streaming 通过 Checkpoint 机制实现端到端的 Exactly-Once 语义，其核心思想是将流处理的关键状态定期持久化到可靠存储中。Checkpoint 目录中保存了以下信息：

- **Kafka Offset**：已消费的每个分区的偏移量，确保故障恢复后不重复消费也不丢失数据。
- **State Store**：窗口聚合的中间状态（各窗口各商品的计数值），恢复后继续累加而非从零开始。
- **Commit Log**：已完成的微批次 ID，防止重复输出。

7.2 容错演示流程

```
# 1. 正常运行流处理和 Producer
make submit-hot-products &
make producer &

# 2. 观察正常输出（若干批次后）

# 3. 模拟 Worker 故障
make fault-stop     # docker stop spark-worker

# 4. 重启 Worker
make fault-start    # docker start spark-worker

# 5. 观察：流处理从 Checkpoint 恢复，继续处理，数据不丢失
```

Listing 5: 容错演示步骤

8 可视化展示

本项目使用 Streamlit 构建了实时仪表盘（`http://localhost:8501`），包含四个页面：

1. **热门商品**：实时显示 Top-10 商品的柱状图和详细表格，10 秒自动刷新。
2. **热搜词**：实时显示 Top-10 搜索关键词的柱状图和详细表格。
3. **流批对比**：展示流处理与批处理的性能指标对比，包括延迟、吞吐量、各阶段耗时柱状图和对比总结表格。
4. **性能指标**：展示流处理的延迟趋势（avg/P95/P99）和吞吐量趋势折线图。

仪表盘支持侧边栏切换页面，以及可配置的自动刷新间隔（5–60 秒），满足演示和日常监控的需求。

图 5 展示了热门商品页面的实际运行效果，左侧边栏提供页面切换和刷新间隔配置，主区域显示 Top-10 商品的柱状图。

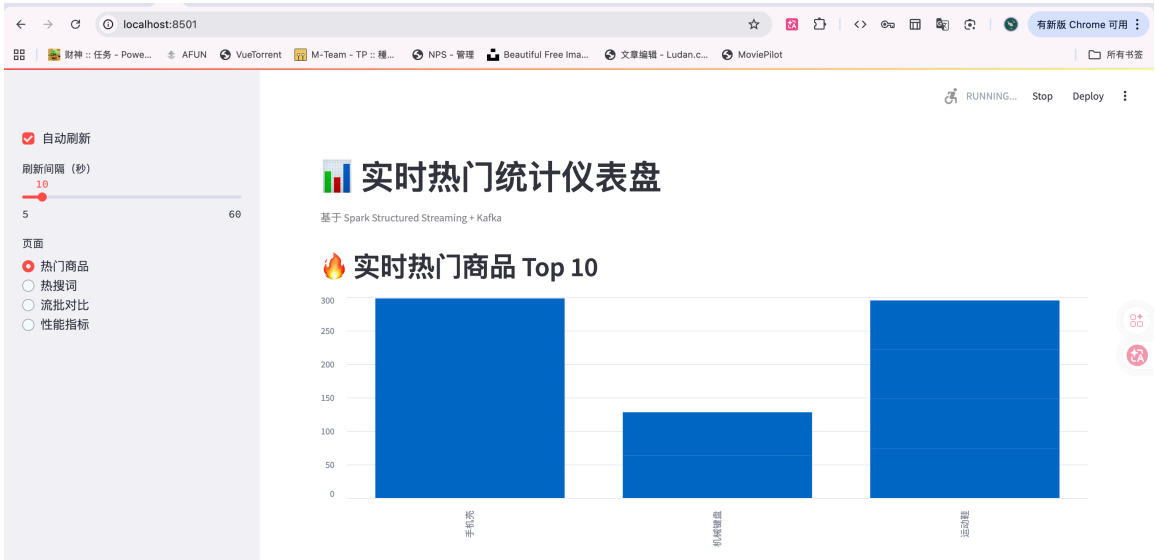

图 5: Streamlit 仪表盘——实时热门商品 Top-10 柱状图

图 6 展示了热门商品的详细数据表格，包含商品名称、分类和浏览次数，可以直观地看到手机壳（75 次）、运动鞋（74 次）、机械键盘（64 次）位列前三。

	product_name	category	view_count
0	手机壳	配件	75
1	手机壳	配件	75
2	手机壳	配件	75
3	运动鞋	运动	74
4	运动鞋	运动	74
5	运动鞋	运动	74
6	手机壳	配件	73
7	运动鞋	运动	73
8	机械键盘	数码	64
9	机械键盘	数码	64

图 6: Streamlit 仪表盘——热门商品详细数据表

图 7 展示了流处理与批处理的性能对比页面，左右两栏分别显示批处理（总记录数 50,000、总耗时 8.25s、吞吐量 6,063 条/秒）和流处理（微批次 10s、延迟 ~10–15 秒）的核心指标。

图 7: Streamlit 仪表盘——流处理 vs 批处理性能对比

9 总结与展望

9.1 项目总结

本项目围绕”基于 Spark 的实时数据分析”这一课题，从系统设计、环境搭建、核心开发到性能评估，完成了一个端到端的实时热门统计系统。项目代码已开源于 GitHub（`https://github.com/Ludan-daye/realtime-hot-stats`），包含完整的源代码、配置文件、测试脚本和文档。主要成果包括：

1. 实现了热门商品 Top-10 和热搜词 Top-10 的**实时统计**，采用滑动窗口和 Watermark 处理乱序数据。
2. 完成了流处理与批处理的**定量对比**，批处理吞吐量达 6,063 条/秒，流处理首条结果延迟 ~10 秒。
3. 通过 Docker 容器化实现了**一键部署**，降低了环境搭建的复杂度。
4. 基于 Checkpoint 实现了**容错恢复**，保证了 Exactly-Once 语义。
5. 通过 Streamlit 仪表盘实现了**实时可视化**，直观展示分析结果。

9.2 技术反思

- **Watermark 阈值的选择**是一个 trade-off：阈值过小会丢弃合理的迟到数据，阈值过大会增加状态存储开销和结果延迟。
- **微批次 vs 连续处理**：本项目使用 Trigger 间隔为 10 秒的微批次模式。Spark 也支持 Continuous Processing 模式（毫秒级延迟），但目前仅支持有限的操作，不支持聚合。
- **JSON vs Parquet**：批处理的 40% 时间花在数据加载上，改用 Parquet 格式可显著提升性能。

9.3 未来展望

1. **多节点集群**：将系统部署到多节点 Spark 集群，测试分布式环境下的可扩展性。
2. **Exactly-Once 端到端验证**：通过注入故障和数据校验，严格验证端到端 Exactly-Once 语义。
3. **实时告警**：在热门统计的基础上增加异常检测（如流量突增告警），扩展系统的实用价值。
4. **生产级存储**：将结果写入 Redis 或时序数据库（如 InfluxDB），替代文件系统输出，支持高并发查询。

参考文献

[1] Zaharia M, Xin R S, Wendell P, et al. Apache Spark: A Unified Engine for Big Data Processing[J]. Communications of the ACM, 2016, 59(11): 56-65.

[2] Armbrust M, Das T, Torres J, et al. Structured Streaming: A Declarative API for Real-Time Applications in Apache Spark[C]. Proceedings of the 2018 International Conference on Management of Data (SIGMOD), 2018: 601-613.

[3] Kreps J, Narkhede N, Rao J. Kafka: A Distributed Messaging System for Log Processing[C]. Proceedings of the NetDB Workshop, 2011.

[4] Dean J, Ghemawat S. MapReduce: Simplified Data Processing on Large Clusters[C]. OSDI, 2004: 137-150.

[5] Apache Spark Documentation: Structured Streaming Programming Guide. https://spark.apache.org/docs/latest/structured-streaming-programming-guide.html

[6] Apache Kafka Documentation. https://kafka.apache.org/documentation/

A 团队分工

成员	负责内容
赵英杰 (D23090600756)	系统架构设计、Spark Streaming 核心开发、Kafka 环境搭建、性能测试
韦多展 (D23090600792)	数据生产者开发、批处理对比分析、可视化仪表盘、报告撰写

B 项目代码结构

```
realtime-hot-stats/
|-- docker-compose.yml          # Kafka Docker 编排
|-- Makefile                    # 常用命令封装
|-- requirements.txt            # Python 依赖
|-- data/
|   |-- products.json           # 50 个商品 (带权重)
|   |-- search_keywords.json    # 40 个搜索关键词 (带权重)
|-- src/
|   |-- producer/
|   |   |-- data_generator.py   # 加权随机数据生成
|   |   |-- kafka_producer.py   # Kafka 生产者
|   |-- streaming/
|   |   |-- common.py           # SparkSession + Schema
|   |   |-- hot_products.py     # 热门商品流处理
|   |   |-- hot_keywords.py     # 热搜词流处理
|   |-- batch/
|   |   |-- batch_analysis.py   # 批处理对比
|   |-- visualization/
|   |   |-- dashboard.py        # Streamlit 仪表盘
|   |-- benchmark/
|       |-- perf_test.py        # 性能测量
|-- output/                     # 运行结果输出
|-- notebooks/
    |-- analysis.ipynb          # 性能分析 Notebook
```

Listing 6: 项目目录结构